

Advanced Topics in Deep Learning

Mostafa Mehdipour Ghazi (ghazi@di.ku.dk)

Pioneer Centre for Artificial Intelligence
Department of Computer Science

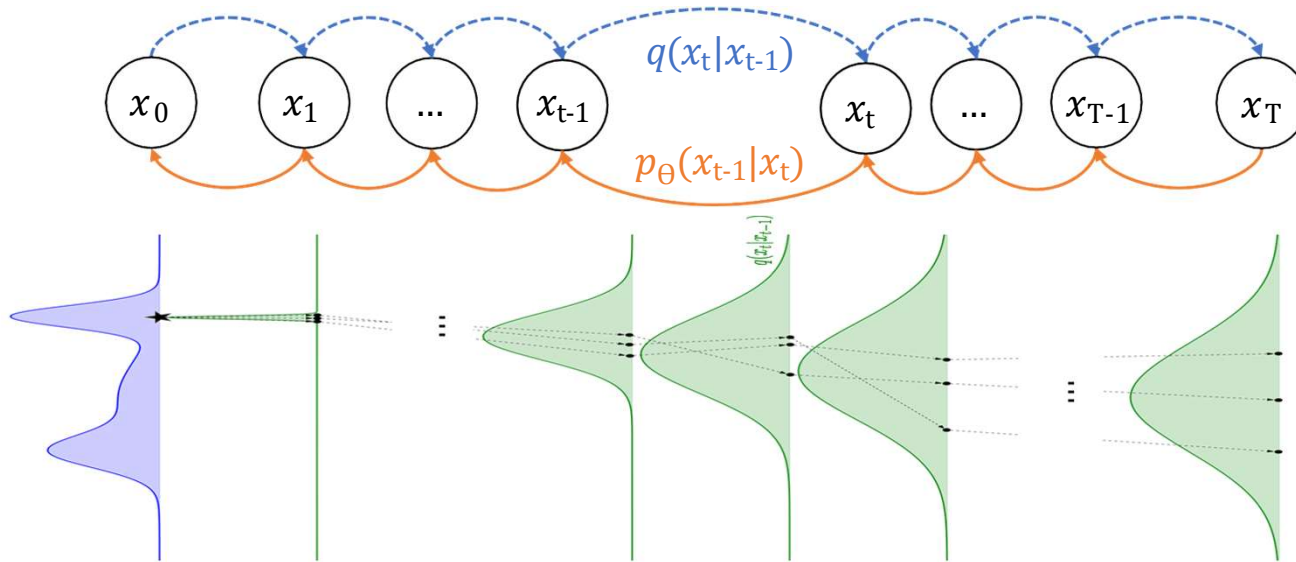
UNIVERSITY OF COPENHAGEN

<https://mostafa-ghazi.github.io/>



Denoising Diffusion Probabilistic Model

- Discrete-time probabilistic diffusion



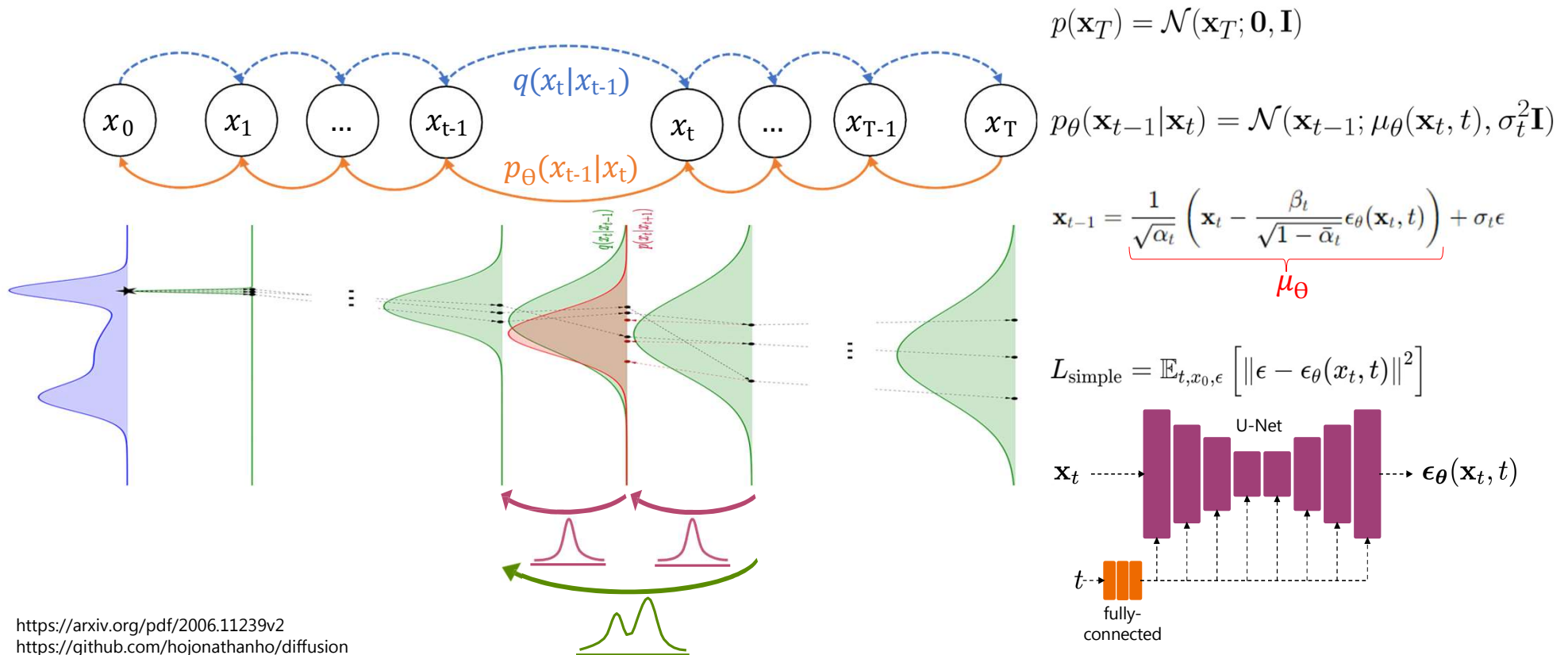
$$\mathbf{x}_t = \sqrt{1 - \beta_t} \mathbf{x}_{t-1} + \sqrt{\beta_t} \mathcal{N}(\mathbf{0}, \mathbf{I})$$

$$q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t} \mathbf{x}_{t-1}, \beta_t \mathbf{I})$$

$$q(\mathbf{x}_T) \approx \mathcal{N}(\mathbf{x}_T; \mathbf{0}, \mathbf{I})$$

DDPM

- Discrete-time probabilistic denoising

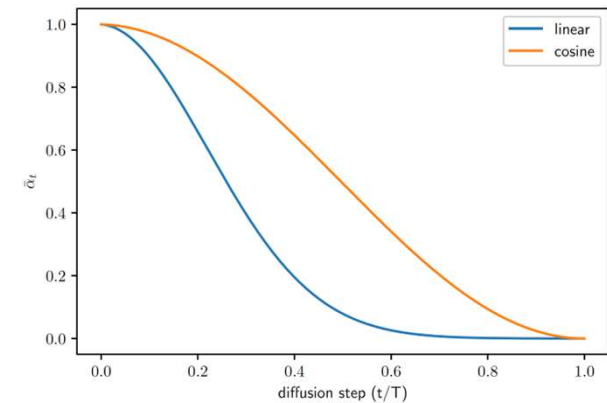


Improved DDPM

- Improved noise schedule
 - Cosine schedule preserves useful signal more gradually than a linear schedule

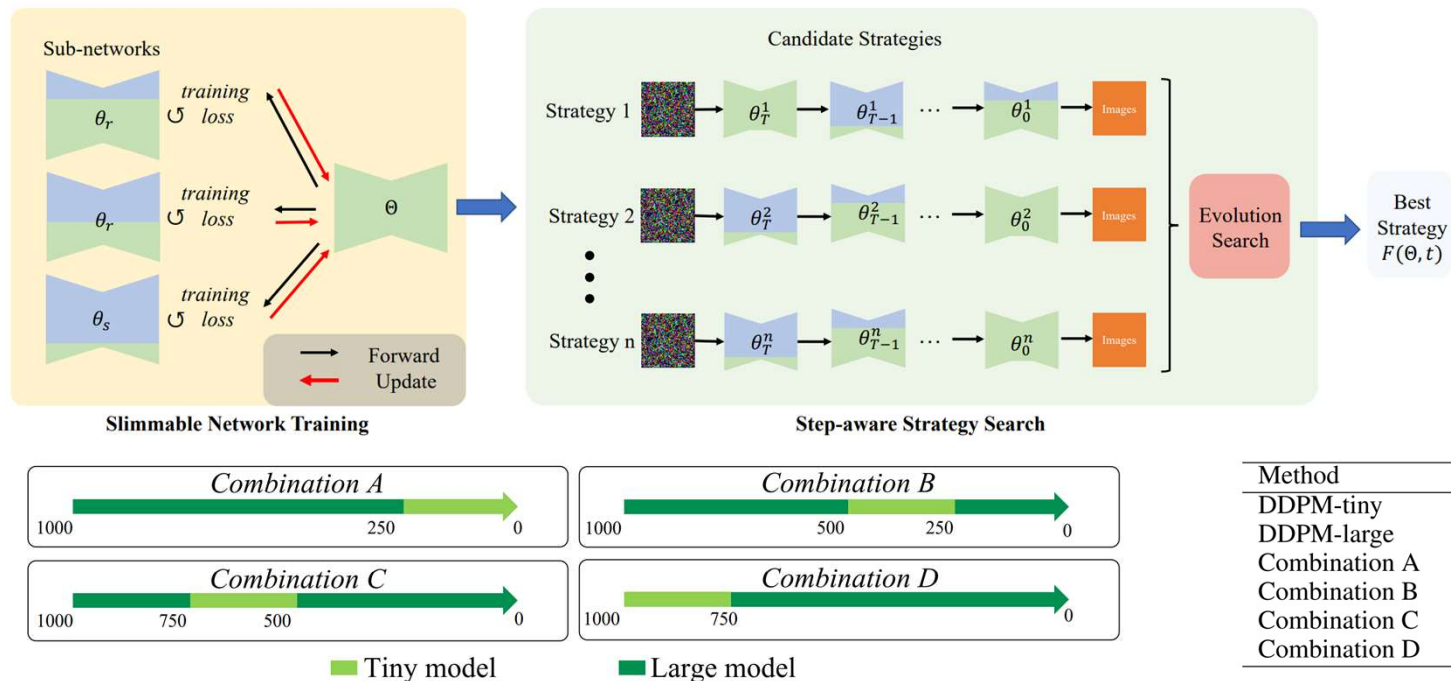
$$\beta_t = 1 - \frac{\bar{\alpha}_t}{\bar{\alpha}_{t-1}} \quad \bar{\alpha}_t = \frac{f(t)}{f(0)}, \quad f(t) = \cos\left(\frac{t/T + s}{1 + s} \cdot \frac{\pi}{2}\right)^2$$

- Reduced sampling steps
 - By learning the reverse-process variance



Denoising Diffusion Step-aware Model

- Improved sampling speed
 - Neural networks whose sizes are adapted based on importance of generative steps



Method	FID	GFLOPs
DDPM-tiny	18.64	0.38
DDPM-large	3.71	12.1
Combination A	10.61	9.20
Combination B	4.38	9.20
Combination C	3.30	9.20
Combination D	3.62	9.20

Denoising Diffusion Implicit Model

- Probabilistic (Markovian) denoising
 - Sampling is stochastic

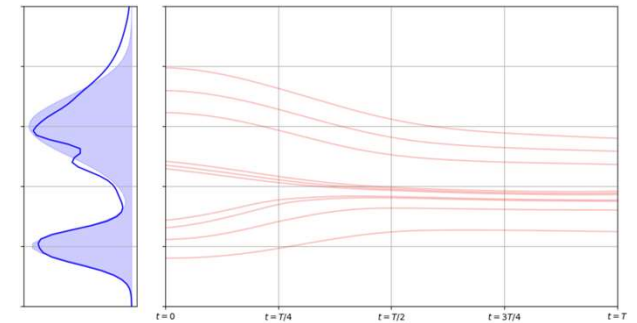
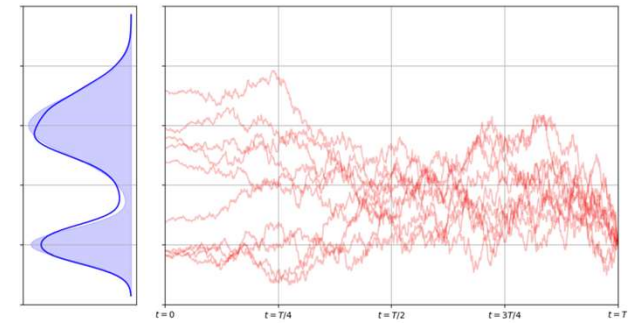
$$p_{\theta}(\mathbf{x}_{t-1}|\mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t-1}; \boldsymbol{\mu}_{\theta}(\mathbf{x}_t, t), \tilde{\beta}_t \mathbf{I})$$

$$\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \boldsymbol{\epsilon}_{\theta}(\mathbf{x}_t, t) \right) + \sqrt{\tilde{\beta}_t} \cdot \mathbf{z}, \quad \mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

- Deterministic (non-Markovian) denoising
 - Stochasticity can be removed from sampling

$$\sigma_t = \eta \sqrt{\frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t}} \sqrt{1 - \frac{\bar{\alpha}_t}{\bar{\alpha}_{t-1}}} \quad \begin{array}{l} \eta = 0: \text{deterministic sampling} \\ \eta > 0: \text{stochastic sampling} \end{array}$$

$$\mathbf{x}_{t-1} = \underbrace{\frac{1}{\sqrt{\alpha_t}} (\mathbf{x}_t - \sqrt{1 - \bar{\alpha}_t} \cdot \boldsymbol{\epsilon}_{\theta}(\mathbf{x}_t, t))}_{\text{Predicted } \mathbf{x}_0} + \underbrace{\sqrt{1 - \bar{\alpha}_{t-1} - \sigma_t^2} \cdot \boldsymbol{\epsilon}_{\theta}(\mathbf{x}_t, t)}_{\text{Direction of } \mathbf{x}_t} + \underbrace{\sigma_t \cdot \mathbf{z}}_{\text{Noise}}$$



Score Matching

- Continuous-time diffusion

- Diffused input

$$\mathbf{x}_t = \sqrt{1 - \beta_t} \mathbf{x}_{t-1} + \sqrt{\beta_t} \mathcal{N}(\mathbf{0}, \mathbf{I})$$

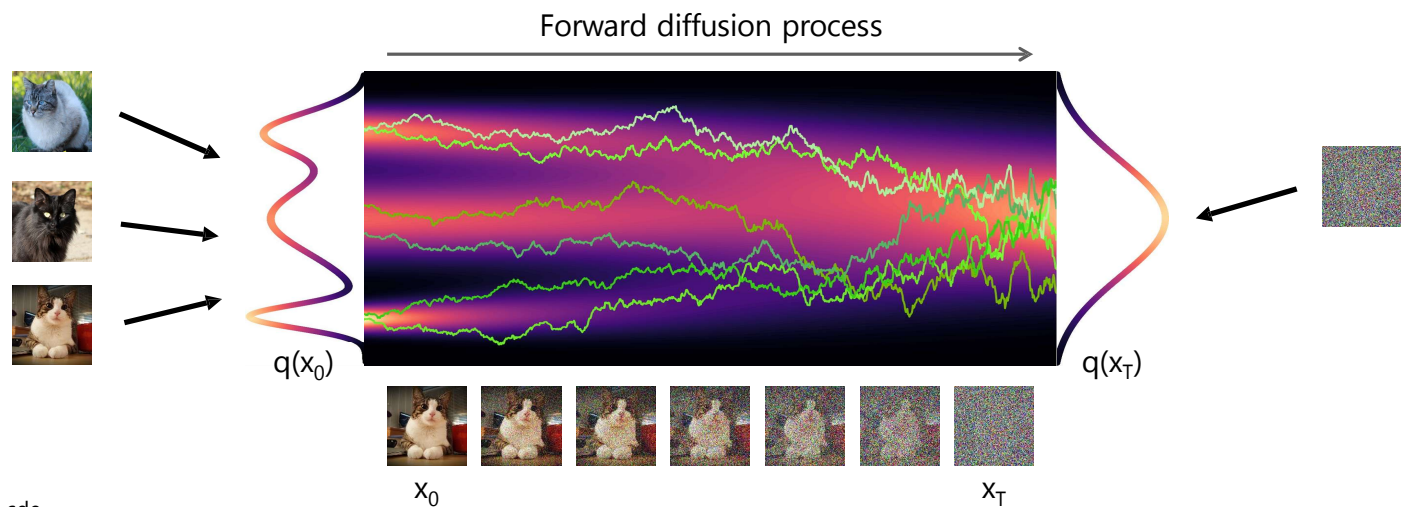
- $\beta_t := \beta(t)\Delta t$

$$= \sqrt{1 - \beta(t)\Delta t} \mathbf{x}_{t-1} + \sqrt{\beta(t)\Delta t} \mathcal{N}(\mathbf{0}, \mathbf{I})$$

- Taylor expansion

$$\approx \mathbf{x}_{t-1} - \frac{\beta(t)\Delta t}{2} \mathbf{x}_{t-1} + \sqrt{\beta(t)\Delta t} \mathcal{N}(\mathbf{0}, \mathbf{I})$$

- Stochastic differential equation $d\mathbf{x} = \mathbf{f}(\mathbf{x}, t)dt + g(t)d\mathbf{w}$

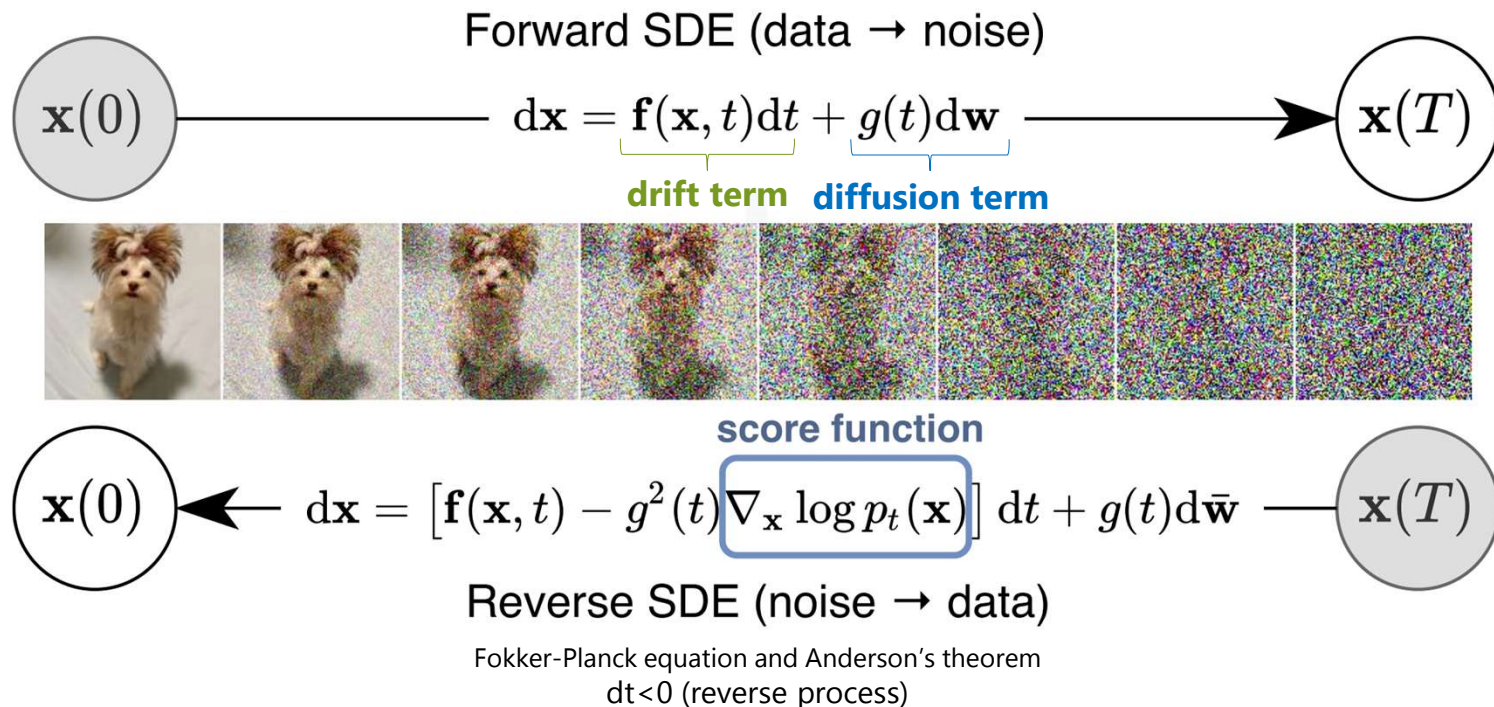


Score Matching

- Deterministic/Ordinary differential equations (ODEs)
 - Deterministic process (**drift term**) $dx / dt = f(x,t)$
 - Analytical (exact) solution is $x(t) = x(0) + \int f(x,s) ds$
 - Numerical (iterative) solution is $x(t+\Delta t) \approx x(t) + f(x(t),t) \Delta t$
- Stochastic differential equations (SDEs)
 - Also includes stochastic process (**diffusion term**) $dx / dt = f(x,t) + \sigma(x,t) \omega(t)$
 - Drift function governs how the data evolves over time (systematic motion)
 - $\omega(t)$ is a Wiener process (Gaussian white noise) for injecting noise at each time step
 - Numerical (iterative) solution is $x(t+\Delta t) \approx x(t) + f(x(t),t) \Delta t + \sigma(x(t),t) \sqrt{\Delta t} N(0,I)$

Score Matching

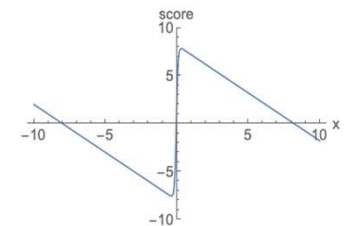
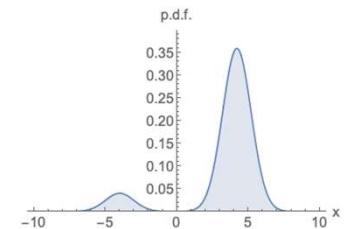
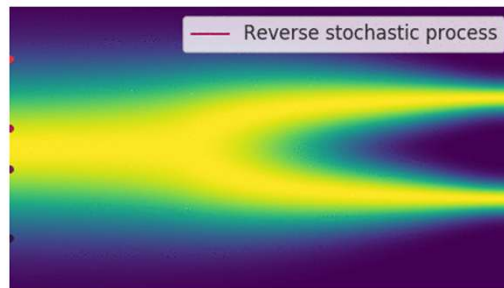
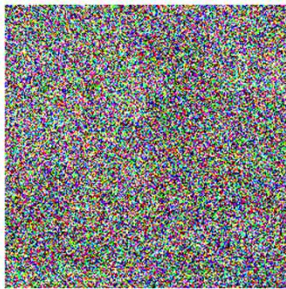
- Train a network to match the score function (diffused data distribution)



Score Matching

- Langevin dynamics with Euler–Maruyama discretization ($\Delta t > 0$)

- forward SDE: $x_{t+\Delta t} = x_t + f(x_t, t)\Delta t + g(t)\sqrt{\Delta t}\epsilon_t \quad \epsilon_t \sim \mathcal{N}(0, I)$
- backward SDE: $x_{t-\Delta t} = x_t - [f(x_t, t) - g(t)^2 \nabla_x \log p_t(x_t)] \Delta t + g(t)\sqrt{\Delta t}\epsilon_t$



Score Matching

- $f(x,t)$ and $g(t)$ are chosen so that $p(x_t|x_0)$ is Gaussian (tractable)

- Variance-Exploding (VE) SDE

- variance explodes from data distribution toward noise
- forward SDE: $f(x,t) = 0$ and $g(t) = \sqrt{d\sigma^2(t)/dt}$

$$dx = \sqrt{\frac{d[\sigma^2(t)]}{dt}} d\mathbf{w} \quad \sigma(t) = \sigma_{\min} \left(\frac{\sigma_{\max}}{\sigma_{\min}} \right)^t \quad dx = \sigma_{\min} \left(\frac{\sigma_{\max}}{\sigma_{\min}} \right)^t \sqrt{2 \log \frac{\sigma_{\max}}{\sigma_{\min}}} d\mathbf{w}, \quad t \in (0, 1] \quad x(t) | x(0) \sim \mathcal{N} \left(x(t); x(0), \sigma_{\min}^2 \left(\frac{\sigma_{\max}}{\sigma_{\min}} \right)^{2t} \mathbf{I} \right)$$



- Variance-Preserving (VP) SDE

- preserves a bounded variance and has $N(0, \mathbf{I})$ as the stationary prior
- forward SDE: $f(x,t) = -0.5\beta(t)x(t)$ and $g(t) = \sqrt{\beta(t)}$

$$dx = -\frac{1}{2}\beta(t)x dt + \sqrt{\beta(t)} d\mathbf{w} \quad \beta(t) = \bar{\beta}_{\min} + t(\bar{\beta}_{\max} - \bar{\beta}_{\min})$$

$$dx = -\frac{1}{2}(\bar{\beta}_{\min} + t(\bar{\beta}_{\max} - \bar{\beta}_{\min}))x dt + \sqrt{\bar{\beta}_{\min} + t(\bar{\beta}_{\max} - \bar{\beta}_{\min})} d\mathbf{w}, \quad t \in [0, 1]$$

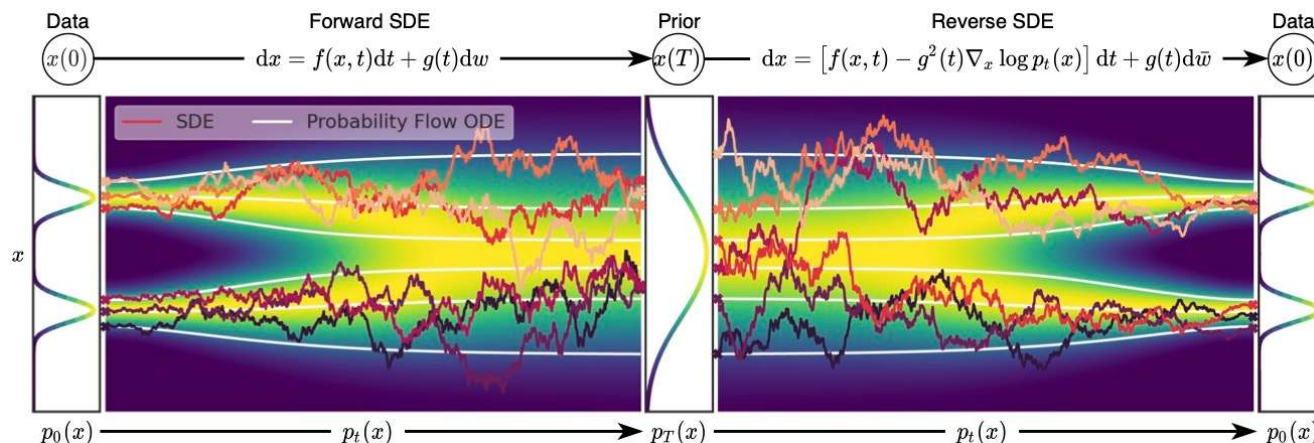
$$x(t) | x(0) \sim \mathcal{N} \left(x(t); e^{-\frac{1}{4}t^2(\bar{\beta}_{\max} - \bar{\beta}_{\min}) - \frac{1}{2}t\bar{\beta}_{\min}} x(0), \mathbf{I} - \mathbf{I} e^{-\frac{1}{2}t^2(\bar{\beta}_{\max} - \bar{\beta}_{\min}) - t\bar{\beta}_{\min}} \right)$$

$$\nabla_{x_t} \log q_t(x_t|x_0) = -\nabla_{x_t} \frac{(x_t - \alpha_t x_0)^2}{2\sigma_t^2} = -\frac{x_t - \alpha_t x_0}{\sigma_t^2}$$

Score Matching

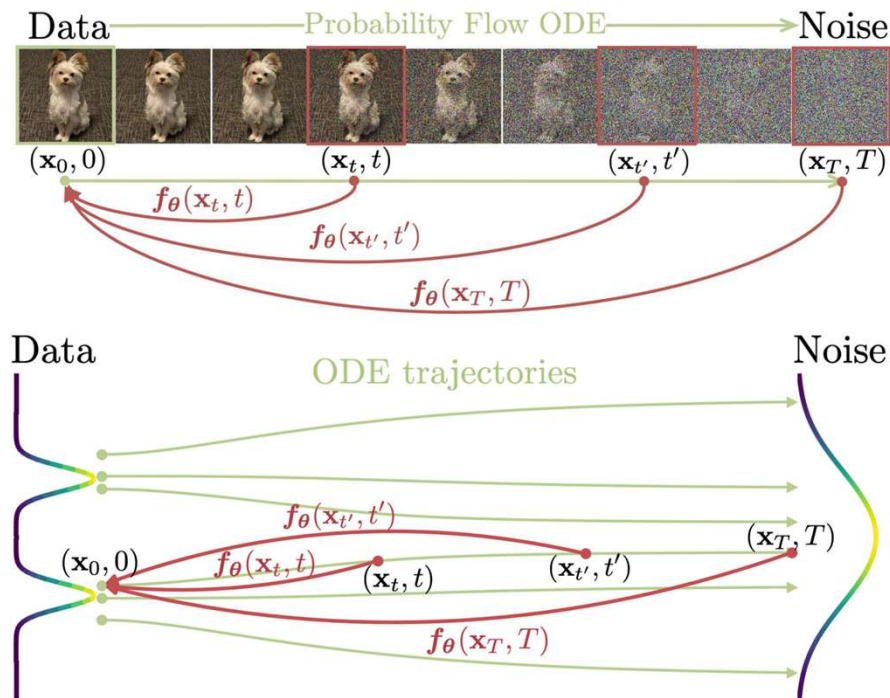
- Probability-flow ODE
 - Use forward SDE to define noisy distributions
 - Train a network to learn the score $\nabla_x \log p_t(x)$
 - Sample using probability-flow ODE (deterministic) instead of reverse SDE (stochastic)

$$dx = \left[f(x, t) - \frac{1}{2} g(t)^2 \nabla_x \log p_t(x) \right] dt$$



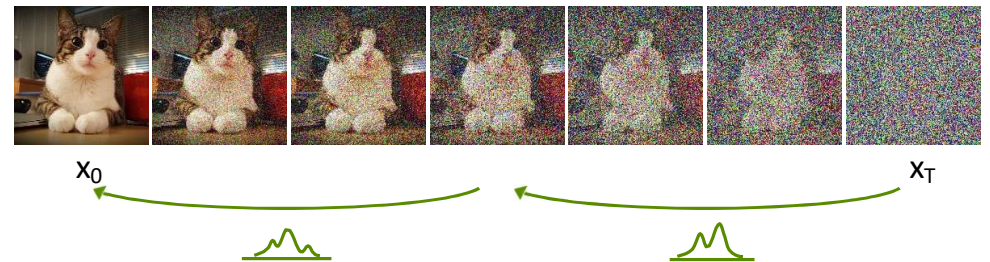
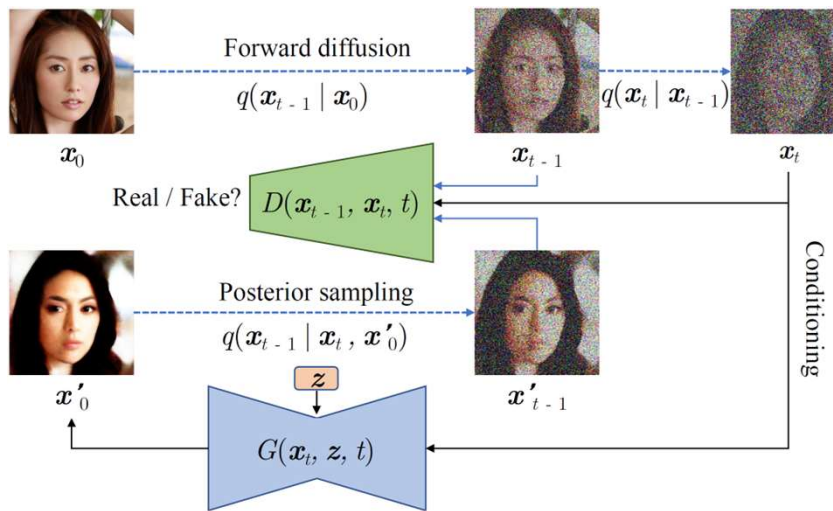
Consistency Models

- Learn a consistency function that maps all points on the same probability-flow ODE trajectory to the same endpoint near the data boundary



Denoising Diffusion GANs

- Tackling the generative learning trilemma with DDGANs
 - An adversarially learned reverse transition so that each denoising step can cover a much larger noise interval.



Guided Diffusion

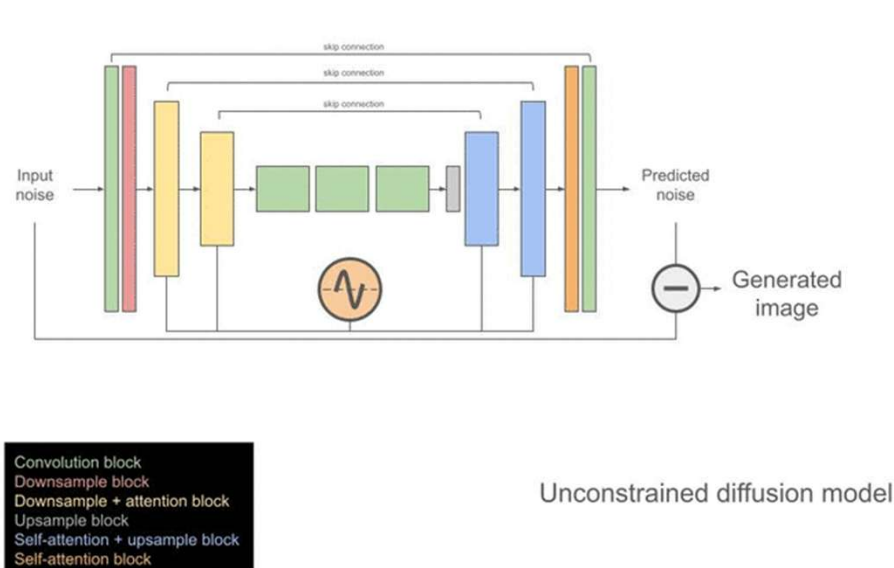
- Reverse process is controlled by additional information or conditions
 - To guide generated image content and improve image quality
- Classifier guidance
 - Train a classifier alongside the diffusion model to predict image class per noise level
 - Use the predictions when sampling to modify the score function and generate an image of the desired class

$$p(x | y) = \frac{p(y | x) \cdot p(x)}{p(y)}$$

$$\underbrace{\nabla_x \log p_\gamma(x | y)}_{\text{Conditional score}} = \underbrace{\nabla_x \log p(x)}_{\text{Unconditional score}} + \underbrace{\gamma \nabla_x \log p(y | x)}_{\text{Classifier score}}$$

Guided Diffusion

- Classifier-free guidance $\nabla_x \log p_\gamma(x | y) = (1 - \gamma) \nabla_x \log p(x) + \gamma \nabla_x \log p(x | y)$
 - Train a diffusion model with the conditioning randomly dropped during training
 - At sampling, combine conditional and unconditional predictions



<https://arxiv.org/pdf/2207.12598>
<https://softwaremill.com/classifier-free-diffusion-model-guidance/>
<https://github.com/jcwang-gh/classifier-free-diffusion-guidance-Pytorch>

Flow Matching

- Conditional flow matching

- Learn a velocity field that moves samples from noise to data

- Using ODE model: $\frac{dx}{dt} = v_\theta(x, t)$

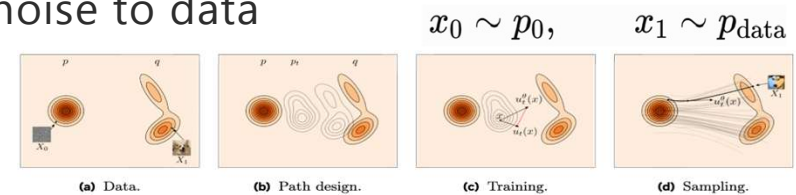
- Conditional vector field u_t is tractable: $\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{t, x_1, x \sim p_t(\cdot|x_1)} [\|v_\theta(x, t) - u_t(x|x_1)\|^2]$

- This provides a transport rule: given x at time t , move it in this direction with this speed

- Rectified flow

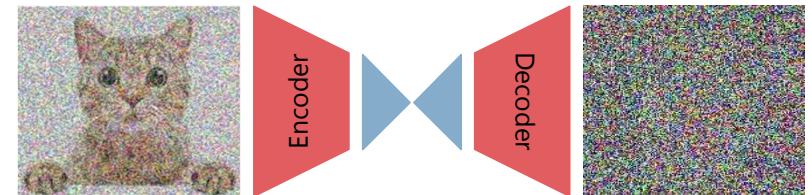
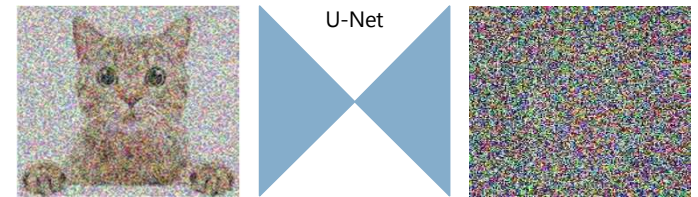
- Make trajectories as straight as possible between noise and data: sample faster

- Straight interpolation between the noise and data: $x_t = (1 - t)x_0 + tx_1$ $u_t = \frac{dx_t}{dt} = x_1 - x_0$
- Euler solver: $x_{k+1} = x_k + \Delta t v_\theta(x_k, t_k)$



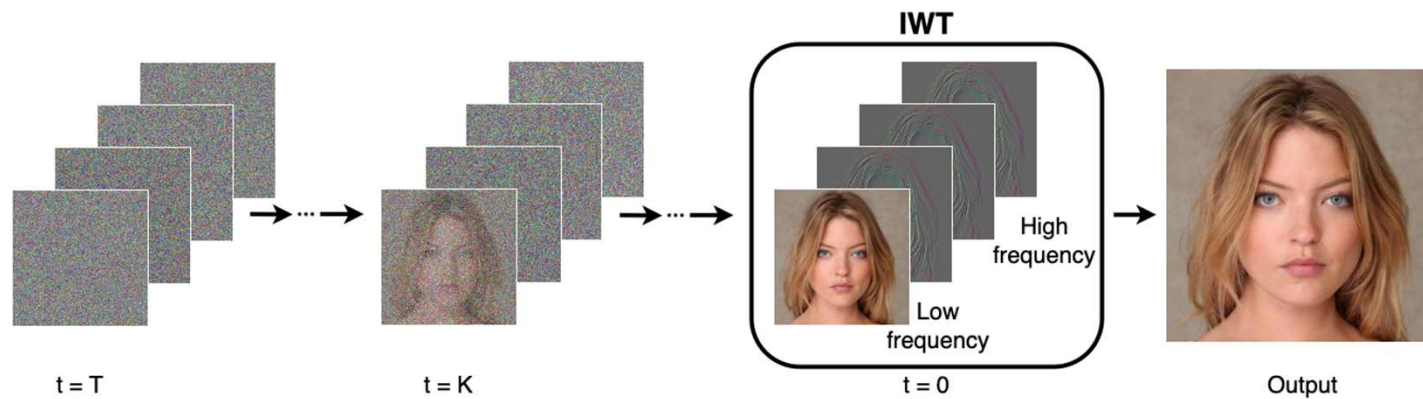
Transform Diffusion

- Pixel space
 - Computationally expensive
- Latent space
 - DWT $\rightarrow$ Generate $\rightarrow$ IDWT
 - Encode $\rightarrow$ Generate $\rightarrow$ Decode
 - Encode $\rightarrow$ Tokenize $\rightarrow$ Generate $\rightarrow$ Decode



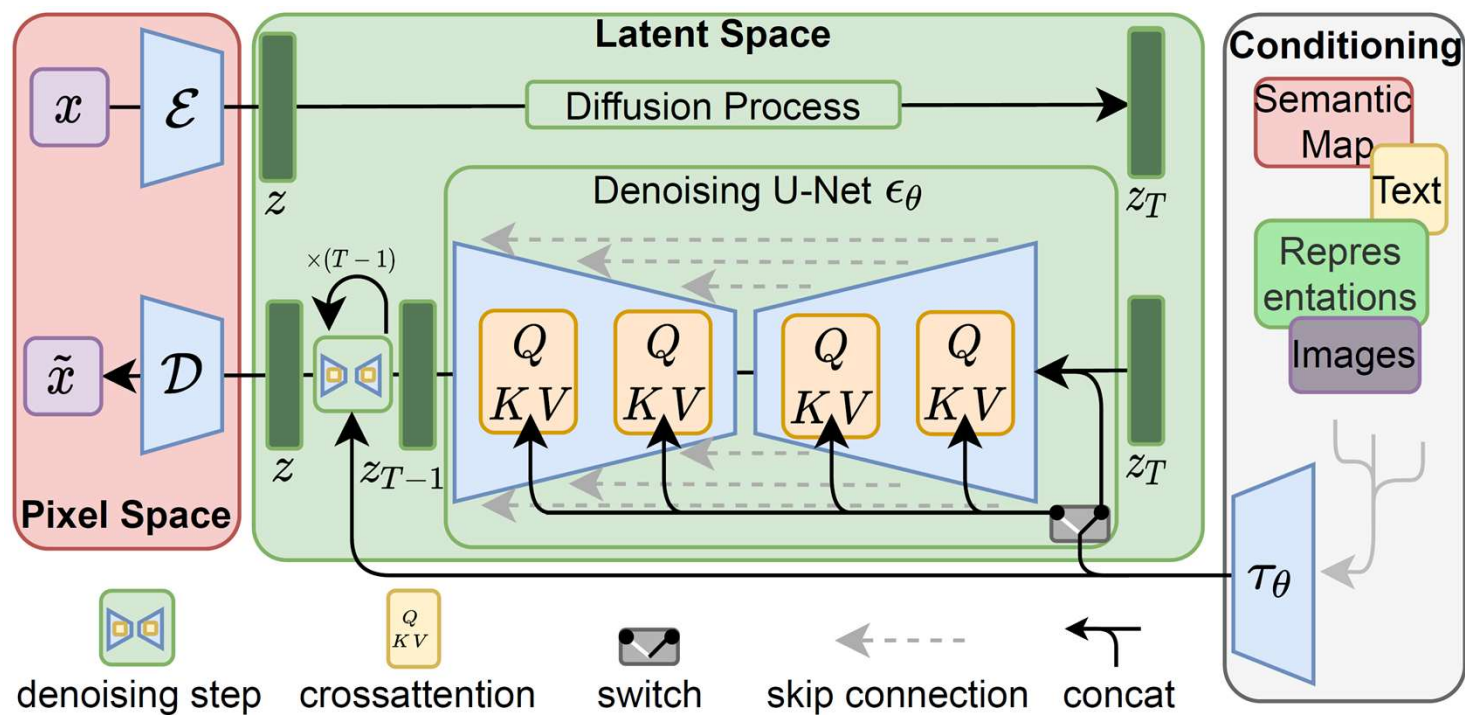
Wavelet Diffusion

- Wavelet diffusion models are fast and scalable image generators
- DWT $\rightarrow$ Diffusion $\rightarrow$ Denoising $\rightarrow$ IDWT



Latent Diffusion (Stable Diffusion)

- High-resolution image synthesis with latent diffusion models

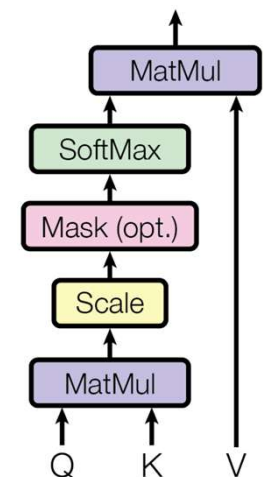


Attention

- Scaled dot-product attention
 - Which parts of the input sequence matter most for each item!
 - Using Queries (Q), Keys (K), and Values (V) to get Outputs (O)

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- QK^T matches a query with all keys to obtain similarity scores
- $\sqrt{d_k}$ scales the scores by the dimensionality of the key vectors
- Softmax converts scores into probabilities (attention weights)
- V is weighted to obtain attention outputs (best matching key-query)



Attention

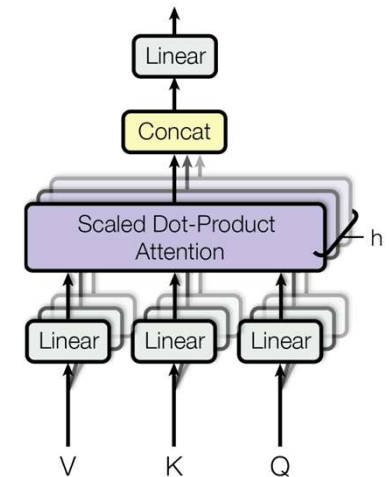
- Multi-head attention

- The model considers different types of relationships in parallel

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

- Input elements linearly projected into multiple Q's, K's, and V's
- Independent attention per projected (Q,K,V) is obtained
- All attention heads are concatenated and linearly projected



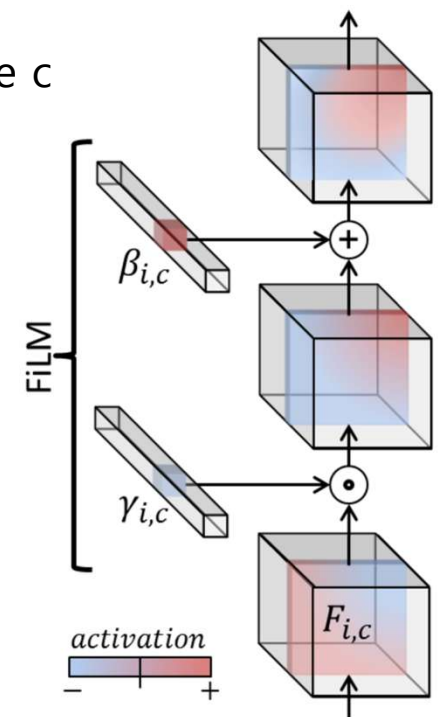
FiLM

- Feature-wise linear modulation (affine transformation)
 - Learning functions f and h from input x , which output γ and β
 - They modulate a network's activations $F_{i,c}$ per input i and feature c

$$\gamma_{i,c} = f_c(\mathbf{x}_i) \quad \beta_{i,c} = h_c(\mathbf{x}_i)$$

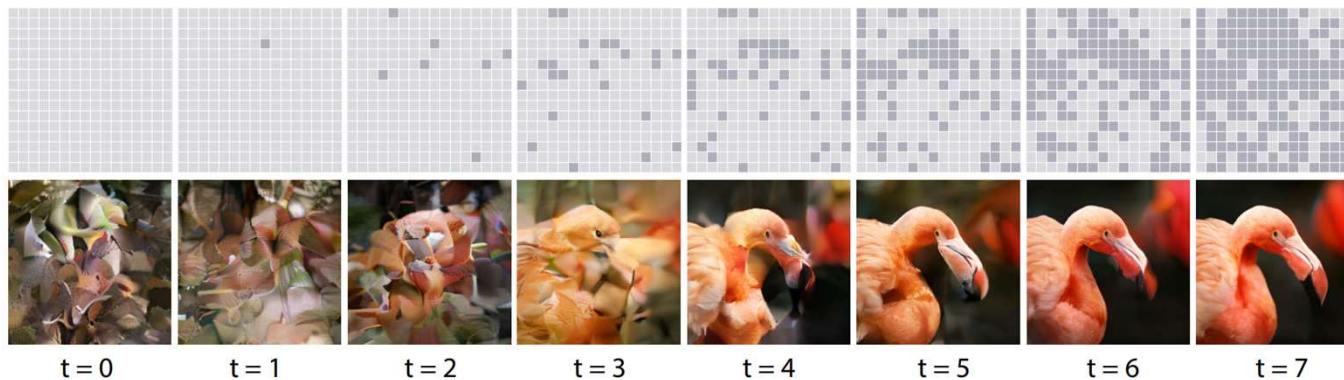
$$FiLM(\mathbf{F}_{i,c} | \gamma_{i,c}, \beta_{i,c}) = \gamma_{i,c} \mathbf{F}_{i,c} + \beta_{i,c}$$

- F can be the latent feature maps
- f and h can be linear fully-connected layers
- x can be the condition vector/image



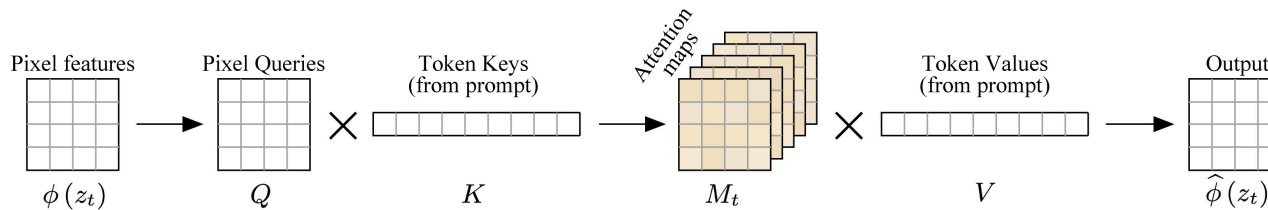
Masked Generative Image Transformer

- MaskGIT
 - Represent images as visual tokens and learn bidirectional masked-token prediction
 - Generation iteratively fills randomly masked tokens in parallel



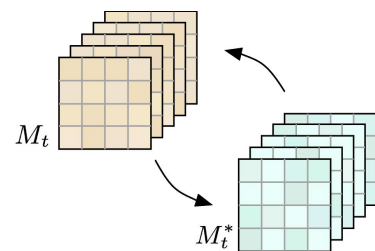
Prompt Editing

- Prompt-to-prompt image editing with cross-attention control

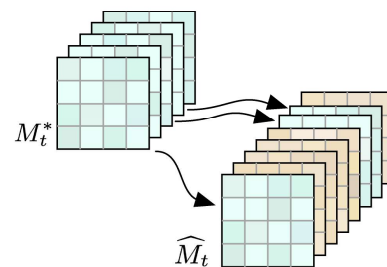


Text to Image Cross Attention

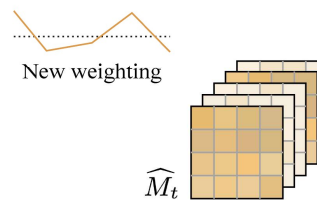
Cross Attention Control



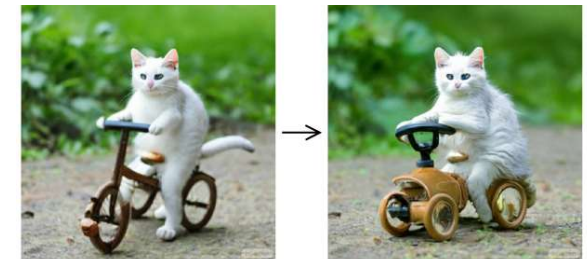
Word Swap



Prompt Refinement



Attention Re-weighting

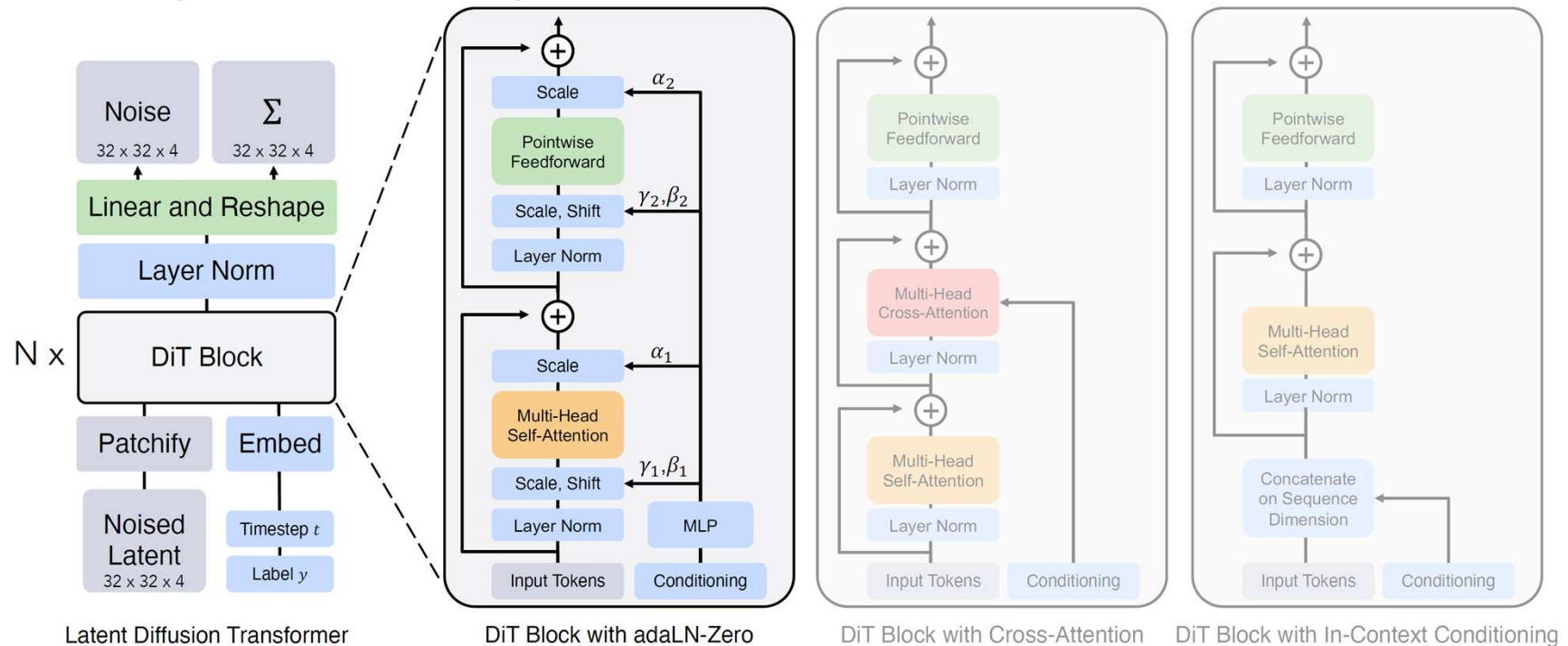


“Photo of a cat riding on a bicycle.”

~~car~~

Diffusion Transformers

- DiT replaces the convolutional U-Net denoiser with salable Transformers operating on latent image patches



Latent Diffusion Transformer

DiT Block with adaLN-Zero

DiT Block with Cross-Attention

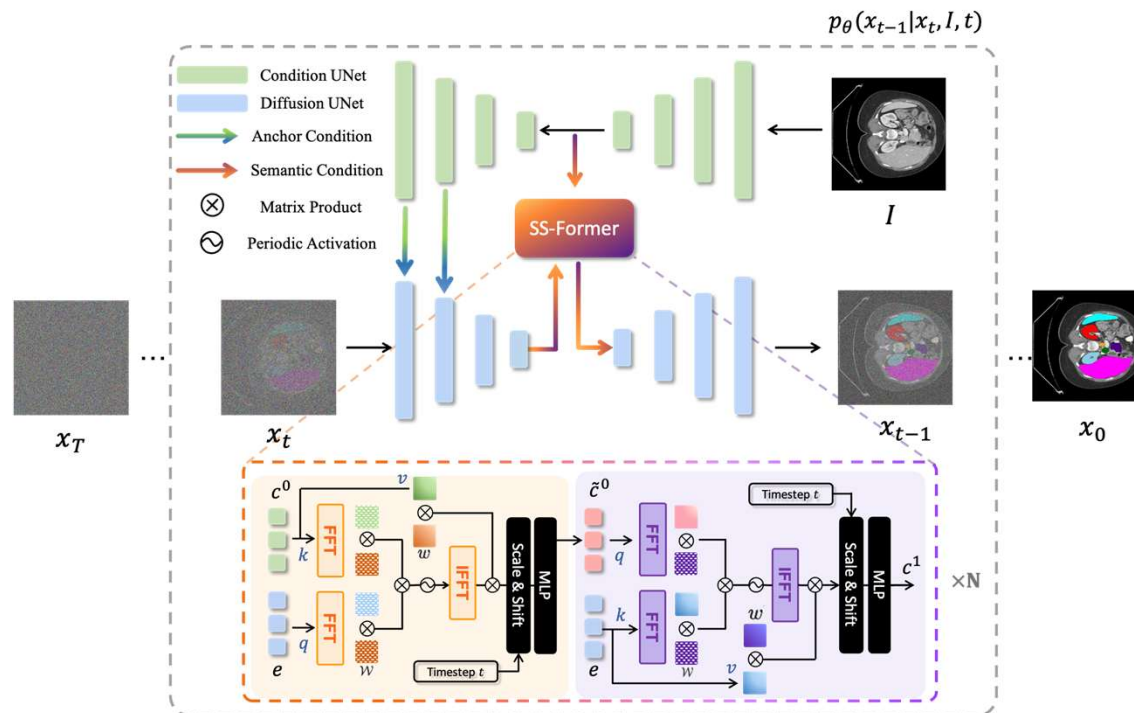
DiT Block with In-Context Conditioning

<https://arxiv.org/pdf/2212.09748v2>

<https://github.com/facebookresearch/DiT>

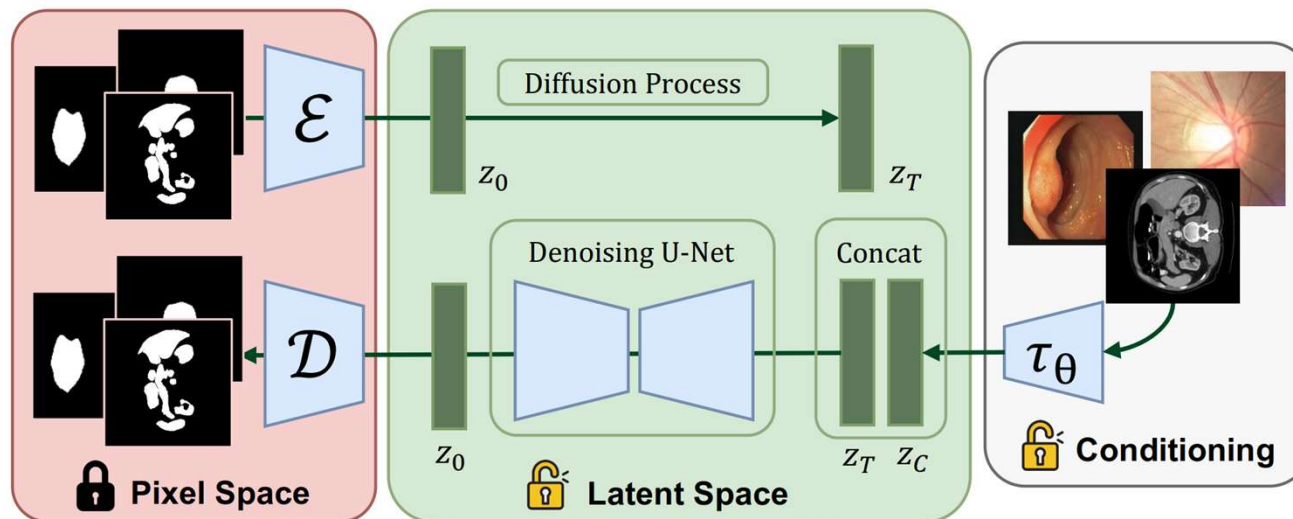
MedSegDiff-V2

- Diffusion-based medical image segmentation with transformer



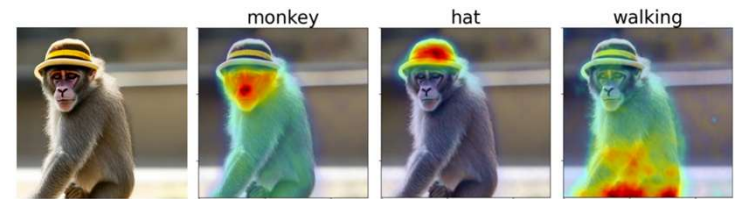
Stable Diffusion Segmentation

- SDSeg uses a latent diffusion to facilitate a single-step reverse process for biomedical Image segmentation



Explainability

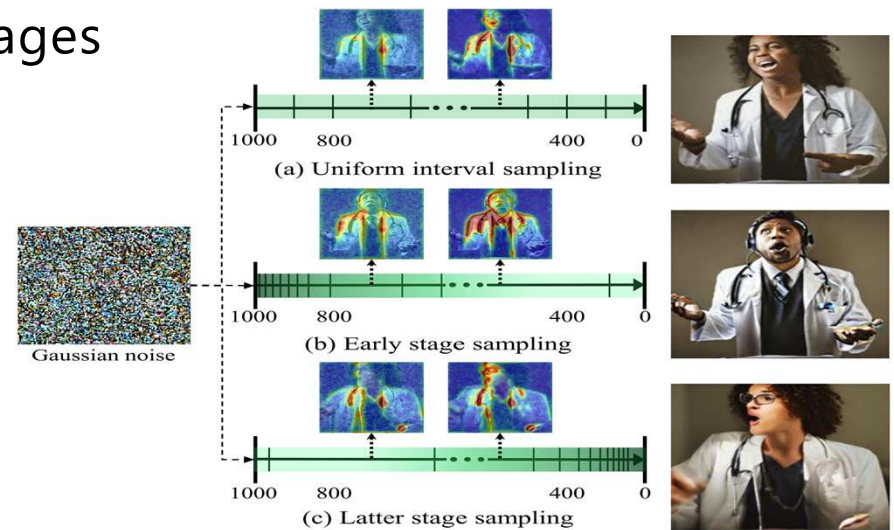
- Cross-attention maps from stable diffusion
 - Upscaling and aggregating the maps to create visual attributions
 - Showing which prompt words influence which image regions



- Attention weights: $A = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right)$
- Attribution map over T steps: $\alpha_j^{(t)} = \frac{1}{L} \sum_{\ell=1}^L \text{reshape}(A_{:,j}^{(t,\ell)})$ $\alpha_j = \frac{1}{T} \sum_{t=1}^T \alpha_j^{(t)}$
 - Q: queries from image latent features (spatial positions)
 - K/V: keys/values from text tokens (prompt embedding)
 - ℓ : cross-attention layers
 - A_{ij} : how much spatial position (latent pixel) i attends to token j

Explainability

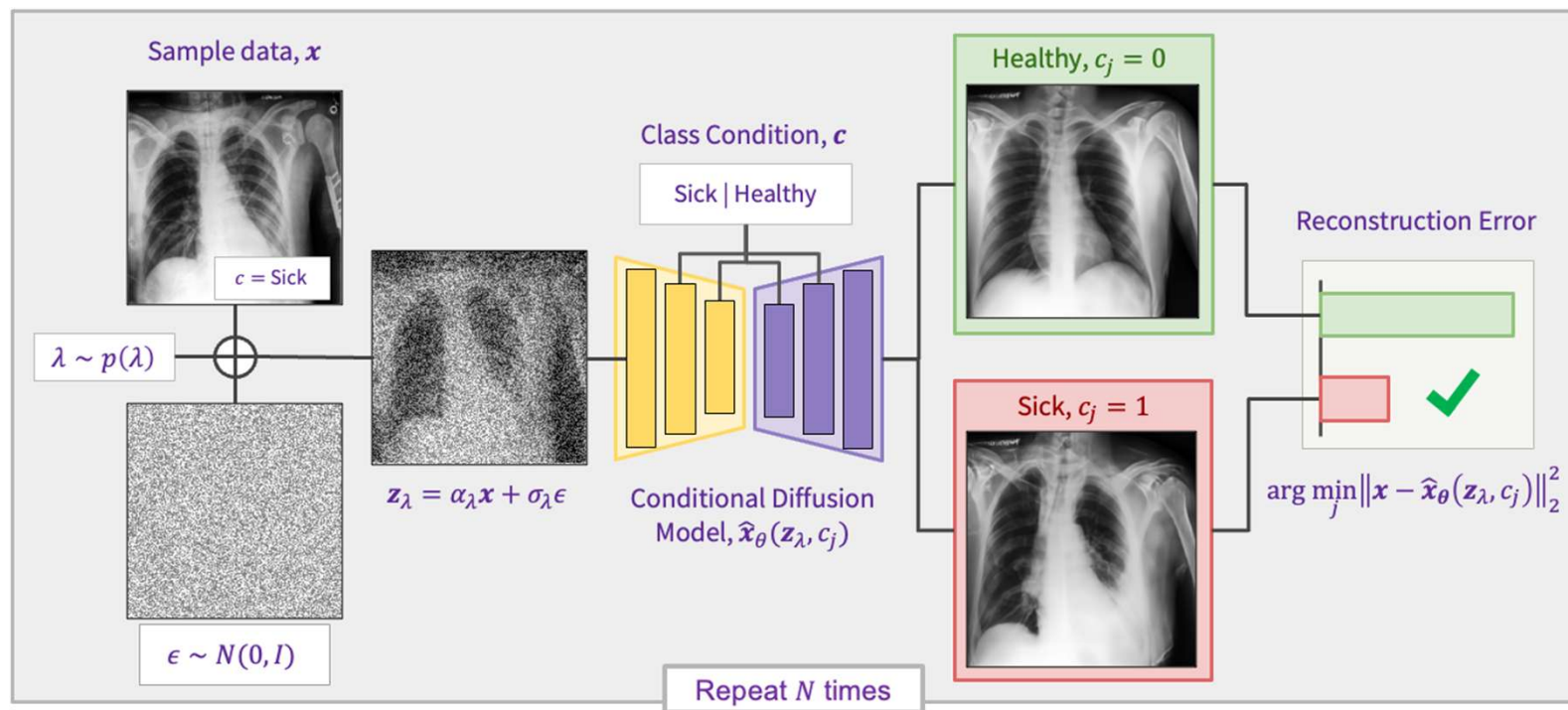
- Saliency map at different sampling stages
 - For images generated from a prompt



- Randomized input sampling explanation (RISE)
 - Perturb input/prompt information and measure output changes
 - Aggregate effects into saliency heatmaps, with SSIM used as a similarity measure

Explainable Diffusion

- Conditional diffusion models provide explainability and uncertainty



Performance Metrics

- Inception Score (IS)
 - Class probabilities $p(y|x)$ is obtained using Inception-v3
 - Marginal class distribution $p(y) = \int p(y|x)p_g(x)dx$ is obtained across generated samples
 - Higher IS is associated with higher quality and diversity

$$\text{IS} = \exp \left(\mathbb{E}_{x \sim p_g} \left[D_{\text{KL}}(p(y|x) \parallel p(y)) \right] \right)$$

- Fréchet Inception Distance (FID)
 - Image features are usually obtained from the Inception-v3 pool3 layer embeddings
 - The mean and covariance are calculated for real and generated data features
 - Lower FID indicates closer distributions

$$\text{FID}(\mathcal{X}_r, \mathcal{X}_g) = \|\mu_r - \mu_g\|_2^2 + \text{Tr}(\Sigma_r + \Sigma_g - 2(\Sigma_r \Sigma_g)^{1/2})$$

<https://arxiv.org/pdf/1606.03498>

<https://arxiv.org/pdf/1706.08500v6>

<https://arxiv.org/pdf/1904.06991v3>

<https://github.com/toshas/torch-fidelity>

Performance Metrics

- Precision and Recall (PRC)
 - Image features are usually obtained from the Inception-v3 pool3 layer embeddings
 - Each real and generated sample gets a k-NN radius in feature space
 - Precision: proportion of generated samples within the k-NN ball of some real sample
 - Recall: proportion of real samples within the k-NN ball of some generated sample
- Downstream Task Scores
 - Classification, segmentation, ...
- Human Testing
 - A/B test, visual Turing test

<https://arxiv.org/pdf/1606.03498>

<https://arxiv.org/pdf/1706.08500v6>

<https://arxiv.org/pdf/1904.06991v3>

<https://github.com/toshas/torch-fidelity>

- MMD does not require fitting a Gaussian distribution to embeddings
 - Inception embeddings of typical images may not be normally distributed.
- MMD estimator is unbiased and sample efficient
 - High-dimensional covariance matrices for few samples can lead to large errors.

Distortion Level	FID	CMMD
Low	1.245	0.355
Low +	1.238	0.360
Low ++	1.232	0.365
Low +++	1.228	0.370
Low ++++	1.225	0.375
Low +++++	1.225	0.380
Low ++++++	1.225	0.385
Low +++++++	1.225	0.390
Low ++++++++	1.228	0.395
Low ++++++++	1.235	0.400
Low ++++++++	1.238	0.405
Low ++++++++	1.260	0.415
Low ++++++++	1.290	0.430
High	1.480	0.460

Thank you!

Any questions?